SOFTWARE ENGINEERING

Practical / Oral Exam Notes

Q1 - Q20 solved step by step · Academic Year 2026-27

Index **What is inside**

Part	Topic	Questions
1	COCOMO / FP / LOC estimation	Q5, Q19, Q18
2	Cyclomatic complexity & testing	Q9, Q10, Q11
3	Project scheduling (Gantt)	Q6
4	Software process models	Q8, Q13, Q14, Q15
5	Data Flow Diagrams	Q7, Q16, Q17
6	UML diagrams	Q1, Q2, Q3, Q4, Q12, Q20

PART 1

Estimation

COCOMO · Function Points · LOC

Memorize **COCOMO constants & formulas**

Type	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Effort E = a × (KLOC)^{b} person-months **Time D** = c × E^{d} months
Staff = E / D **Cost** = E × cost per PM
Calculator trick: K$^{1.05}$ = K × K$^{0.05}$. Use the x^{y} key. KLOC means thousands (20 KLOC = 20, not 20000).

Q5 Basic COCOMO - 20 KLOC, Organic, Rs 30,000/PM

Step 1 **Write given data**

KLOC = 20, a = 2.4, b = 1.05, c = 2.5, d = 0.38, rate = Rs 30,000 per person-month.

Step 2 **Effort**

E = 2.4 × 20$^{1.05}$
20$^{1.05}$ = 20 × 20$^{0.05}$ = 20 × 1.1616 = 23.23
E = 2.4 × 23.23 = **55.76 PM**

Step 3 **Development time**

D = 2.5 × (55.76)$^{0.38}$
(55.76)$^{0.38}$ = 4.608
D = 2.5 × 4.608 = **11.52 months**

Step 4 **Average staff**

N = E / D = 55.76 / 11.52 = 4.84 ≈ **5 persons**

Step 5 **Cost**

Cost = 55.76 × 30,000 = **Rs 16,72,800**

```
output
Effort = 55.76 PM
Time = 11.52 months
Staff = 5
Cost = Rs 16,72,800
```

Q19 Smart Parking - 28 KLOC, Organic

Step 1 **Given**

KLOC = 28, Organic: a=2.4, b=1.05, c=2.5, d=0.38. Productivity = 600 LOC per PM.

Step 2 **Effort (COCOMO)**

28$^{1.05}$ = 28 × 28$^{0.05}$ = 28 × 1.1813 = 33.08
E = 2.4 × 33.08 = **79.4 PM**

Step 3 **Time**

D = 2.5 × (79.4)$^{0.38}$ = 2.5 × 5.271 = **13.2 months**; Staff = 79.4/13.2 = **6 persons**

Step 4 **Effort (LOC method)**

Size = 28 KLOC = 28,000 LOC
Effort = Size / Productivity = 28,000 / 600 = **46.67 PM**

Compare: COCOMO (79.4) is higher than the LOC method (46.7) because COCOMO's exponent 1.05 makes effort grow faster than size. LOC method assumes constant productivity.

```
output
COCOMO: E = 79.4 PM, D = 13.2 months
LOC method: E = 46.67 PM
```

Q18 Online Pharmacy - Function Points + COCOMO (32 KLOC, Semi-detached)

Memorize **FP weights - all complexity levels**

Function type	Simple	Average	Complex
External Input (EI)	3	**4**	6
External Output (EO)	4	**5**	7
External Inquiry (EQ)	3	**4**	6
Internal Logical File (ILF)	7	**10**	15
External Interface File (EIF)	5	**7**	10

Question says **average complexity**, so use the bold Average column. UFP = sum of (count × weight). AFP = UFP × VAF.

Step 1 **Unadjusted FP (UFP)**

UFP = (18×4) + (14×5) + (10×4) + (8×10) + (5×7)
= 72 + 70 + 40 + 80 + 35 = **297**

Step 2 **Adjusted FP (AFP)**

AFP = UFP × VAF = 297 × 1.05 = **311.85**

Step 3 **Effort with constants given in question (a=2.4, b=1.05, c=2.5, d=0.38)**

$32^{1.05}$ = 32 × 1.1892 = 38.05; E = 2.4 × 38.05 = **91.3 PM**

D = 2.5 × $91.3^{0.38}$ = 2.5 × 5.558 = **13.9 months**; Staff = 6.6 ≈ 7

Step 4 **Same project with standard Semi-detached constants (3.0, 1.12, 2.5, 0.35)**

$32^{1.12}$ = 48.5; E = 3.0 × 48.5 = **145.5 PM**

D = 2.5 × $145.5^{0.35}$ = 2.5 × 5.713 = **14.3 months**

Step 5 **Suitability discussion**

Semi-detached suits teams of mixed experience and medium complexity (payments, prescription check, inventory, delivery tracking). Typical size is 50-300 KLOC; 32 KLOC is small, near the organic range (2-50 KLOC), so the estimate may be on the higher side. Because of integration and regulation needs, Semi-detached is still a reasonable choice.

```
output
UFP = 297, AFP = 311.85
Effort = 91.3 PM (given constants) / 145.5 PM (standard)
```

PART 2

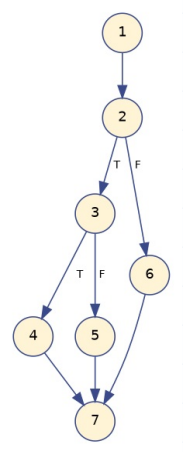
Cyclomatic Complexity

Basis path · White-box testing

Memorize **Three ways to find V(G)**
1) V = E - N + 2 (edges, nodes) **2)** V = P + 1 (P = decision points: if, while, for, case) **3)** V = number of regions (include outer region).
Steps: number the statements → draw CFG → find V → list V independent paths → one test case per path. `break` is only an edge, not a decision.

Q9 Basis Path Testing - Even / Odd

Node	Statement
1	read n
2	if (n > 0)
3	if (n % 2 == 0)
4	print Even
5	print Odd
6	print Invalid
7	End (join)



Step 1 **Count**

N = 7, E = 8, decisions P = 2 (n>0 and n%2==0)

Step 2 **Cyclomatic complexity**

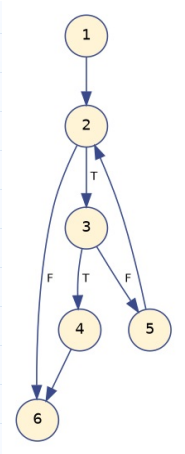
V = E - N + 2 = 8 - 7 + 2 = **3** | V = P + 1 = 2 + 1 = **3**

Step 3 **Independent paths and test cases**

Path	Nodes	Test n	Output
P1	1-2-6-7	-1	Invalid
P2	1-2-3-4-7	4	Even
P3	1-2-3-5-7	3	Odd

Q10 White-box Testing - Prime check

Node	Statement
1	read n; i = 2; isPrime = true
2	while (i <= n/2)
3	if (n % i == 0)
4	isPrime = false; break
5	i = i + 1
6	print isPrime



Step 1 **Count**

N = 6, E = 7, predicates P = 2 (while, if)

Step 2 **Cyclomatic complexity**

V = 7 - 6 + 2 = **3** | V = 2 + 1 = **3**

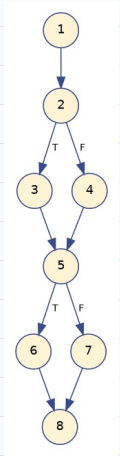
Step 3 **Paths and tests (integer division)**

Path	Nodes	Test n	Result
P1	1-2-6	3 (loop skipped)	isPrime = true
P2	1-2-3-4-6	4 (divisible, break)	false
P3	1-2-3-5-2-6	7 (no divisor)	true

Q11 Basis Path Testing - Max and sign

Note: Assumption: the second `if (max > 0)` is a separate statement after the first if-else.

Node	Statement
1	read a, b
2	if (a > b)
3	max = a
4	max = b
5	if (max > 0)
6	print Positive
7	print Non-positive
8	End (join)



Step 1 **1. CFG**

Drawn above: N = 8 nodes, E = 9 edges.

Step 2 **2. Cyclomatic complexity**

V = 9 - 8 + 2 = **3** | P + 1 = 2 + 1 = **3**

Step 3 **3. Independent paths**

P1: 1-2-3-5-6-8 P2: 1-2-4-5-6-8 P3: 1-2-3-5-7-8
(each new path uses at least one new edge)

Step 4 4. Test cases

Path	a	b	Expected
P1	5	3	Positive
P2	2	4	Positive
P3	-1	-5	Non-positive

Extra (not independent): a=-5, b=-2 gives Non-positive via 1-2-4-5-7-8.

PART 3

Project Scheduling

Activities · Dependencies · Gantt chart

Q6 Student Management System - Schedule

Step 1 Identify activities and durations

ID	Activity	Duration	Depends on
A	Requirement analysis	2 weeks	-
B	System and database design	2 weeks	A
C	Student registration module	2 weeks	B
D	Attendance module	2 weeks	C
E	Examination results module	2 weeks	C
F	Fee management module	2 weeks	C
G	Integration	1 week	D, E, F
H	Testing	2 weeks	G
I	Deployment and training	1 week	H

Step 2 Gantt chart (12 weeks)

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
A	■	■										
B			■	■								
C					■	■						
D							■	■				
E							■	■				
F							■	■				
G									■			
H										■	■	
I												■

Blue = critical path, green = parallel tasks.

Step 3 Critical path

A → B → C → D (or E, F) → G → H → I = 2+2+2+2+1+2+1 = **12 weeks**. D, E, F run in parallel.

PART 4

Software Process Models

Case study → keywords → model → diagram

Memorize **Which model to pick?**

Keyword in case study	Model
Clear, fixed requirements, no change, secure	Waterfall
Requirements unclear, needs user feedback	Prototype
High risk, security, big costly project	Spiral
Changing requirements, fast delivery, sprints	Agile (Scrum)
Deliver in parts / increments	Incremental

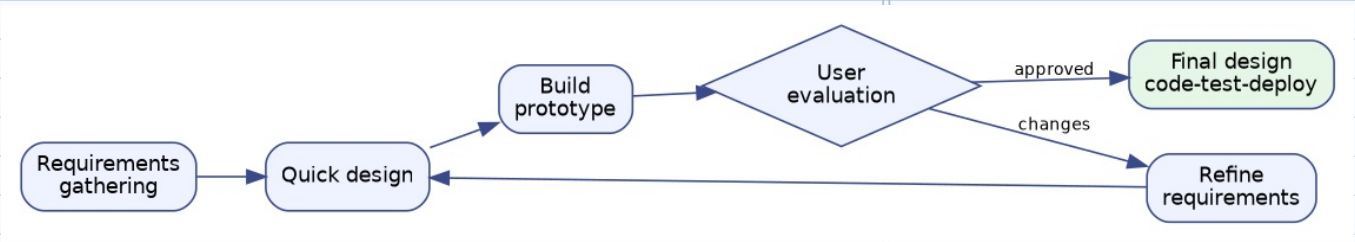
Q8 Online Examination System

Step 1 **Key facts**

Requirements **may change** with **feedback** from faculty and students.

Step 2 **Choose**

Prototyping model - build a quick version, collect feedback, refine until accepted.



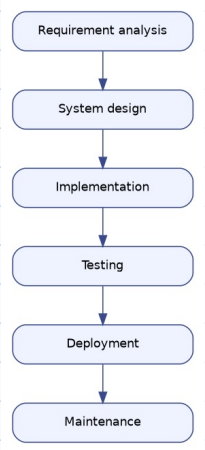
Q13 Hospital Management System

Step 1 **Key facts**

Clearly defined requirements, **no major change**, secure and reliable.

Step 2 **Choose**

Waterfall model - sequential phases with documentation and review at each stage.



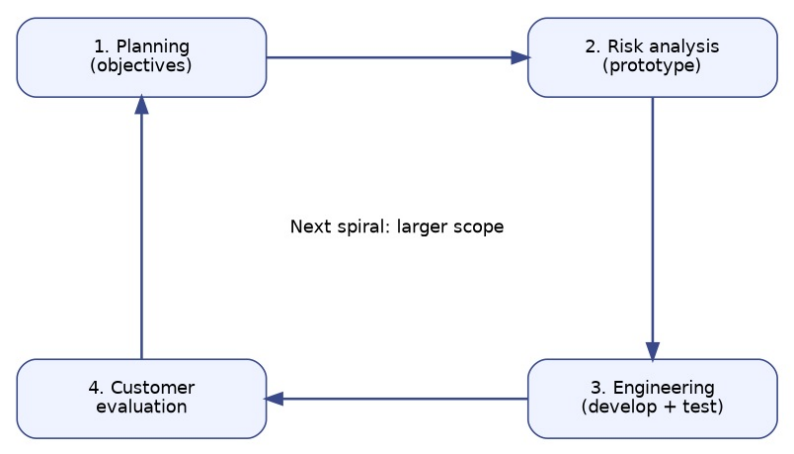
Q14 Mobile Banking Application

Step 1 **Key facts**

Financial transactions → **high security, risk management, extensive testing**.

Step 2 **Choose**

Spiral model - every loop: plan, analyse risk, build/test, customer evaluation. Radius of the spiral = cost, angle = progress.



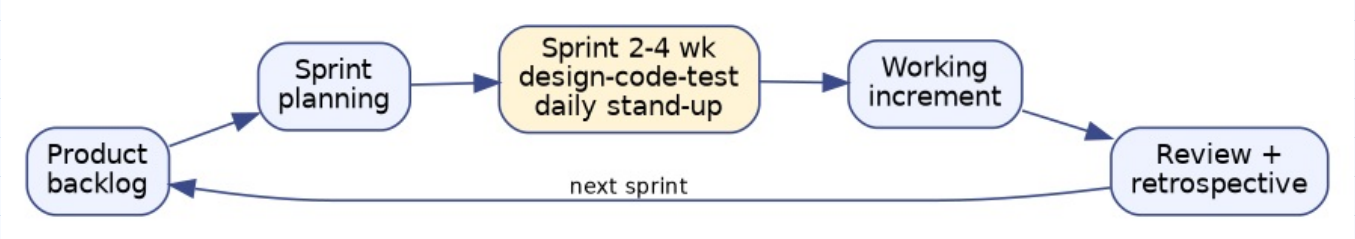
Q15 AI-based Smart Traffic Management

Step 1 **Key facts**

AI + IoT, **evolving requirements**, needs experimentation.

Step 2 **Choose**

Agile (Scrum) - short sprints (2-4 weeks), working increment each sprint, frequent feedback.



PART 5

Data Flow Diagrams

Level 0 (context) and Level 1

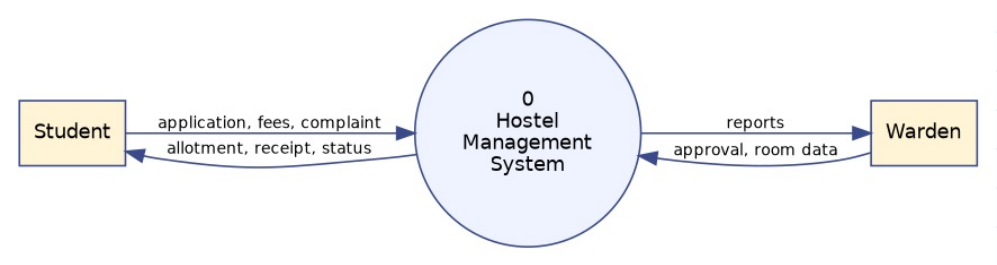
Memorize **DFD rules**
Entity = rectangle (outside the system) · Process = circle, numbered 1.0, 2.0 · Data store = open box/cylinder D1, D2 · Flow = labelled arrow. Level 0 has ONE process. Data cannot flow store-to-store or entity-to-entity directly; it must pass through a process. Level 1 must keep the same inputs/outputs as Level 0.

Q7 Hostel Management System

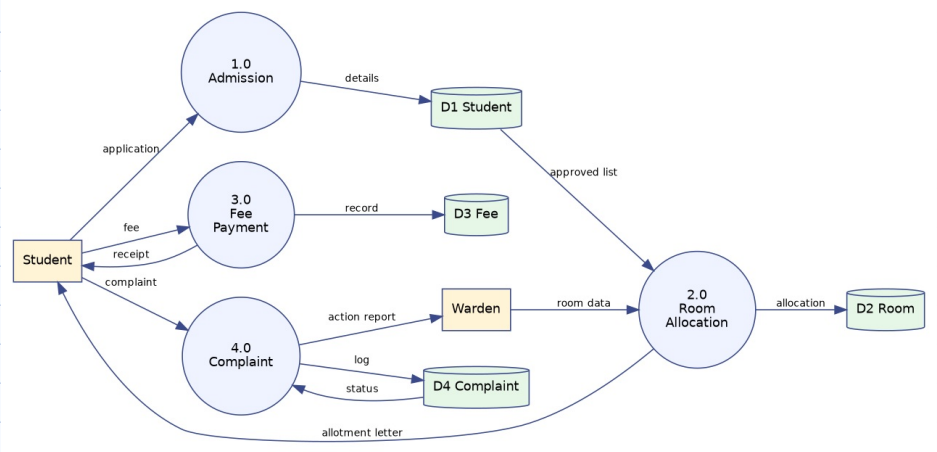
Step 0 **Identify**

Entities: Student, Warden. Processes: Admission, Room Allocation, Fee Payment, Complaints. Stores: Student, Room, Fee, Complaint.

Step 1 **Level 0 (context) diagram**



Step 2 **Level 1 diagram**

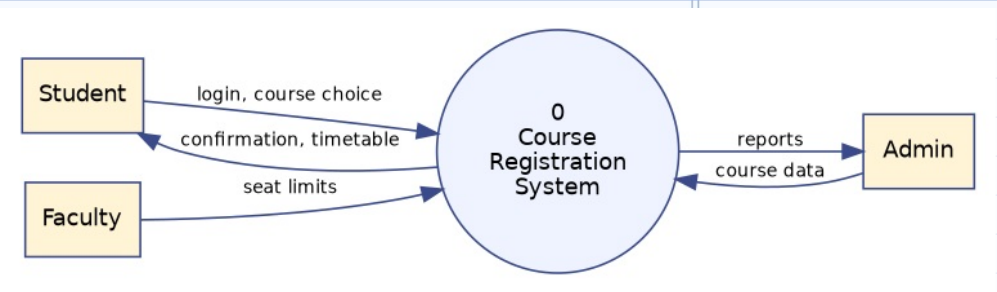


Q16 Course Registration System

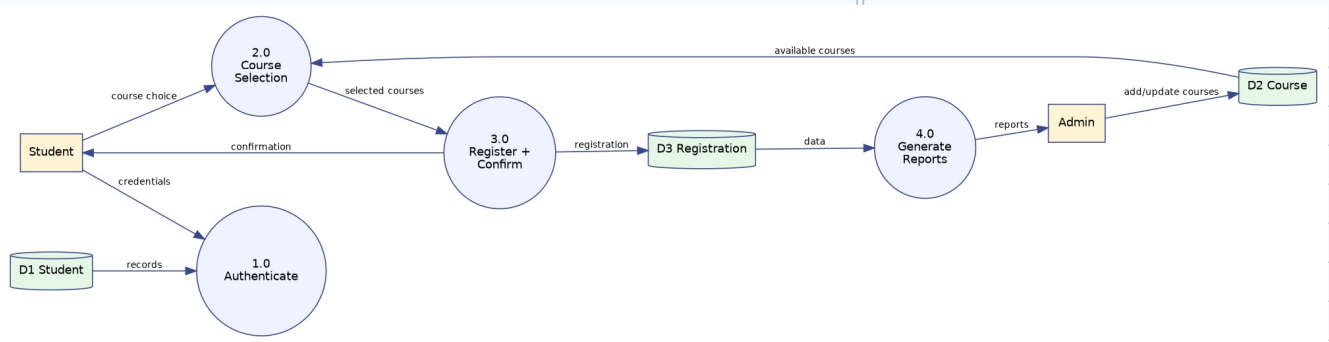
Step 0 **Identify**

Entities: Student, Admin, Faculty. Processes: Authenticate, Course selection, Register and confirm, Reports. Stores: Student, Course, Registration.

Step 1 **Level 0 (context) diagram**



Step 2 **Level 1 diagram**

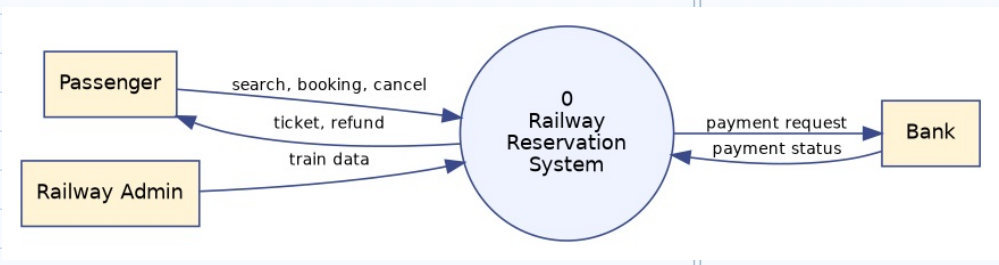


Q17 Railway Reservation System

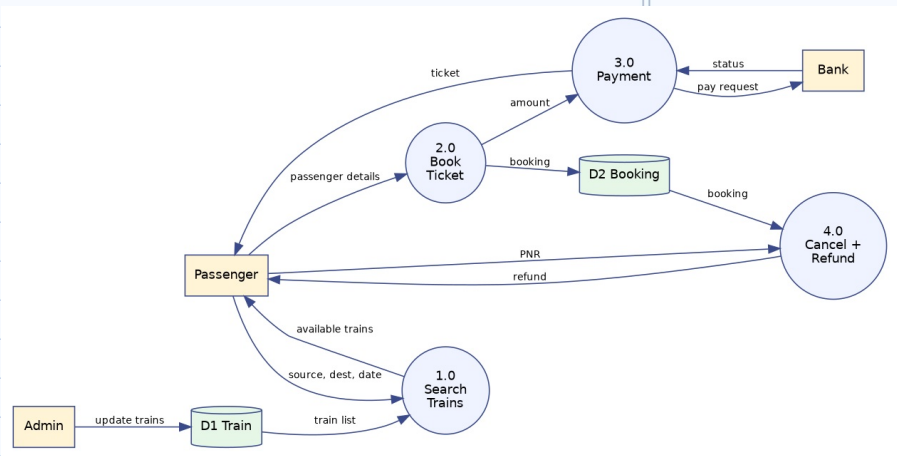
Step 0 **Identify**

Entities: Passenger, Bank, Admin. Processes: Search, Book, Payment, Cancel and refund. Stores: Train, Booking.

Step 1 **Level 0 (context) diagram**



Step 2 **Level 1 diagram**



PART 6

UML Diagrams

Use case · Class · State · Activity · Component

Memorize **Symbols**
Use case: actor + oval + boundary box, «include» = mandatory, «extend» = optional. Class: box with Name / Attributes / Methods; hollow triangle = inheritance, line with 1..* = association, hollow diamond = aggregation, filled diamond = composition. State: rounded box, black dot = start, bullseye = end. Activity: rounded actions, diamond = decision, bars = fork/join.

Q1 Use Case - Library Management System

Step 1 **Actors**

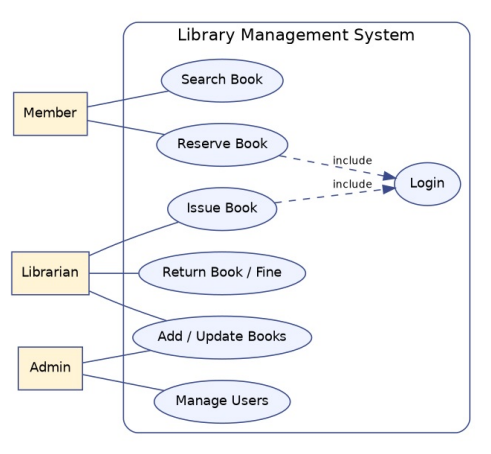
Member, Librarian, Admin.

Step 2 **Use cases**

Login, Search book, Reserve book, Issue book, Return book / fine, Add-update books, Manage users.

Step 3 **Draw**

Actors outside the system boundary, ovals inside, lines for association, dashed «include» for Login.



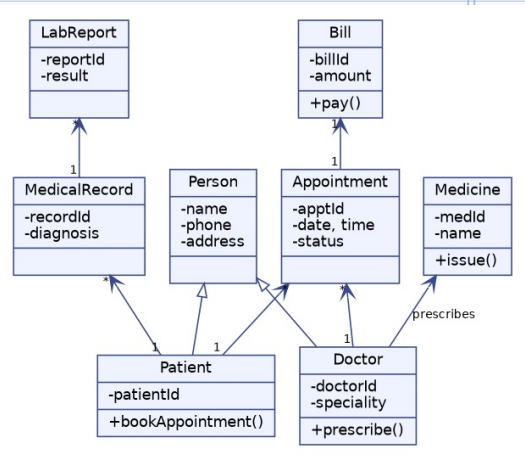
Q2 Class Diagram - Hospital Management System

Step 1 **Classes**

Person (parent), Patient, Doctor, Appointment, Bill, MedicalRecord, LabReport, Medicine/Pharmacy.

Step 2 **Relationships**

Patient and Doctor inherit Person; Patient 1..* Appointment; Doctor 1..* Appointment; Appointment 1..1 Bill; Patient 1..* MedicalRecord; MedicalRecord 1..* LabReport; Doctor prescribes Medicine.



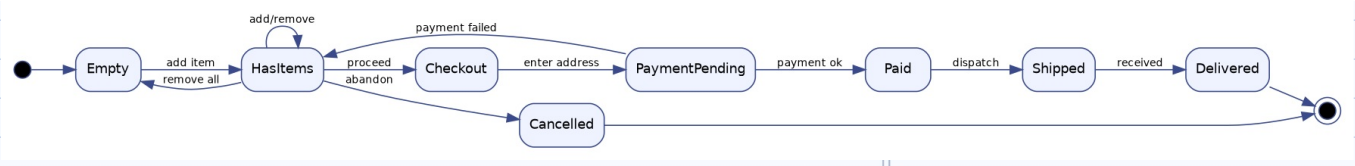
Q3 Statechart - Online Shopping Cart

Step 1 **States**

Empty, HasItems, Checkout, PaymentPending, Paid, Shipped, Delivered, Cancelled.

Step 2 **Transitions**

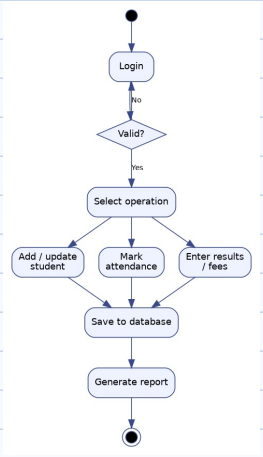
Triggered by events: add item, remove item, proceed, payment success/failed, dispatch, receive, abandon.



Q4 Activity Diagram - Student Management System

Step 1 Flow

> Start → Login → validate (decision) → choose operation → add/update student, mark attendance, enter results/fees → save to database → generate report → End.



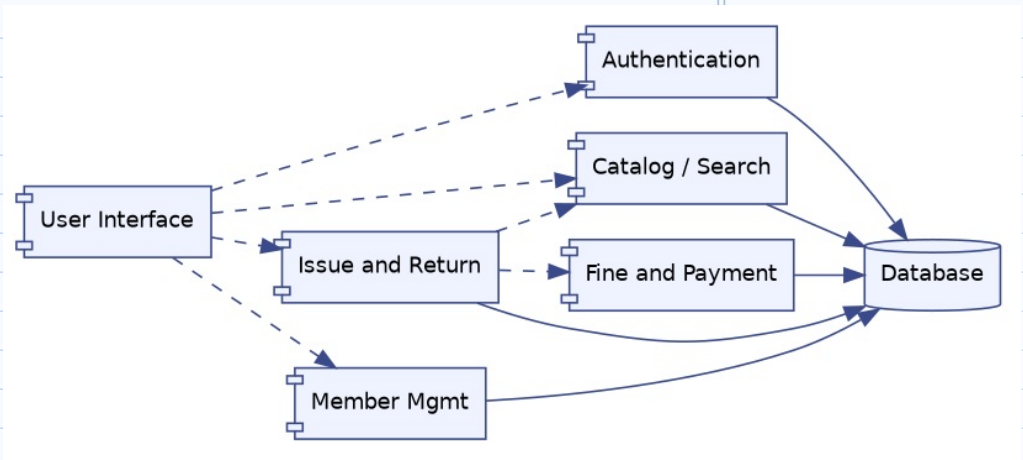
Q12 Component Diagram - Library Management System

Step 1 Components

> User Interface, Authentication, Catalog/Search, Issue and Return, Member Management, Fine and Payment, Database.

Step 2 Dependencies

> Dashed arrows show which component uses another.



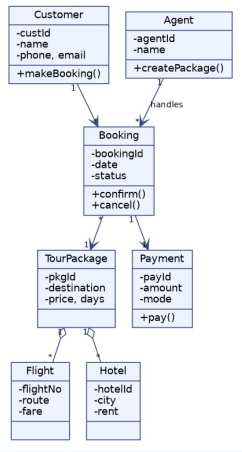
Q20 Class Diagram - Travel Agency System

Step 1 Classes

> Customer, Agent, Booking, TourPackage, Flight, Hotel, Payment.

Step 2 Relationships

> Customer 1..* Booking; Agent 1..* Booking; Booking *..1 TourPackage; TourPackage aggregates Flight and Hotel; Booking 1..1 Payment.



~ all the best for tomorrow ~